Investigación del Método de Jacobi para Valores Propios

CE1111 - Análisis Numérico para Ingeniería

Estudiantes: Joaquin Ignacio Ramírez Sequeira Joseph Stif Piedra Montero

Semestre II, 2026

1. Explicación del método de Jacobi para Valores Propios

El método de Jacobi es un algoritmo iterativo del álgebra lineal numérico empleado para calcular todos los valores propios y sus respectivos vectores propios simultáneamente. El núcleo del método consiste en aplicar una secuencia de transformaciones ortogonales, conocidas como *rotaciones de Givens*, para anular sistemáticamente los elementos fuera de la diagonal principal de la matriz [1, 2].

En cada iteración, el método identifica el elemento fuera de la diagonal con el mayor valor absoluto, denotado como a_{pq}. A continuación, se construye una matriz de rotación plana P que actúa sobre las filas y columnas p y q con un ángulo θ elegido específicamente para hacer que las nuevas entradas a_{pq} y a_{qp} sean exactamente cero. Al aplicar estas rotaciones de forma repetida, la suma de los cuadrados de los elementos fuera de la diagonal converge a cero de forma cuadrática, diagonalizando la matriz [2].

2. Condiciones para la aplicación del método

Para que el método clásico de Jacobi sea aplicable y garantice convergencia matemática, la matriz original $A \in \mathbb{R}^{n\times n}$ debe ser estrictamente simétrica ($A = A^T$, lo que implica que $a_{ij} = a_{ji}$) [3].

Para los efectos específicos de los problemas analizados en este contexto práctico, se asume y requiere una condición adicional más fuerte: la matriz debe ser simétrica semidefinida positiva. Esta propiedad asegura teóricamente que todos los valores propios que el método va a aproximar serán números reales y no negativos ($\lambda_i \geq 0$).

3. Obtención de las aproximaciones

El método genera una sucesión de matrices $A^{(k)}$ mediante la transformación de similitud [1]:

$$A^{(k+1)} = P_k^T A^{(k)} P_k$$

Dado que P_k es ortogonal ($P_k^T = P_k^{-1}$), cada matriz $A^{(k+1)}$ es semejante a $A^{(k)}$, por lo que todas conservan los mismos valores propios.

- **Valores Propios:** Cuando el algoritmo alcanza el criterio de parada (los elementos fuera de la diagonal son despreciables), la matriz $A^{(k)}$ se ha convertido efectivamente en una matriz diagonal D. Las aproximaciones de los *valores propios* se leen directamente de las entradas de la diagonal principal de D [3].

- **Vectores Propios:** Se inicializa una matriz $V^{(0)} = I$ (identidad). En cada paso, esta matriz acumula las rotaciones multiplicándose por la derecha: $V^{(k+1)} = V^{(k)} P_k$. Al finalizar, las columnas de la matriz V contienen los correspondientes *vectores propios normalizados* [2]. La columna j de V es el vector propio asociado al valor propio en la entrada d_{jj} de la matriz diagonal.

4. Pseudocódigo del Algoritmo

Algorithm 1 Método de Jacobi para Valores Propios (Adaptado)

Require: Matriz SPSD $A \in \mathbb{R}^{n \times n}$, iterMax, tolerancia tol.
Ensure: Valores propios $\lambda^{(k)}$, Vectores propios $V^{(k)}$, error er_k, iteraciones k, convergencia $conv$.

$V \leftarrow I_n$
$k \leftarrow 0$, $conv \leftarrow 0$, $er_k \leftarrow \infty$
$\lambda^{(prev)} \leftarrow$ Extraer la diagonal de A y ordenarla de mayor a menor.
while $k < iterMax$ **do**
    Buscar el elemento máximo fuera de la diagonal: $\max_{p \neq q} |A_{pq}|$ y guardar índices p, q
    if $A_{pp} == A_{qq}$ **then**
        $\theta \leftarrow \frac{\pi}{4}$
    else
        $\theta \leftarrow \frac{1}{2} \arctan\left(\frac{2A_{pq}}{A_{pp} - A_{qq}}\right)$
    end if
    $c \leftarrow \cos(\theta)$, $s \leftarrow \sin(\theta)$
    Construir matriz de rotación $P \leftarrow I_n$
    $P_{pp} \leftarrow c$, $P_{qq} \leftarrow c$, $P_{pq} \leftarrow -s$, $P_{qp} \leftarrow s$
    $A \leftarrow P^T A P$ ▷ Transformación de similitud
    $V \leftarrow VP$ ▷ Acumulación de vectores propios
    Extraer $\lambda^{(k)} \leftarrow \text{diag}(A)$
    Obtener índices para ordenar $\lambda^{(k)}$ de mayor a menor y ordenar $\lambda^{(k)}$
    $er_k \leftarrow \|\lambda^{(k)} - \lambda^{(prev)}\|_2$ ▷ Cálculo del error normado
    $k \leftarrow k + 1$
    if $er_k < tol$ **then**
        $conv \leftarrow 1$
        break
    end if
    $\lambda^{(prev)} \leftarrow \lambda^{(k)}$
end while
Reordenar las columnas de V usando los índices del ordenamiento descendente de $\lambda^{(k)}$.
Normalizar cada columna j de V para garantizar $\|v_j\|_2 = 1$.
return $\lambda^{(k)}, V, er_k, k, conv$

5. Ejemplo completo de aplicación (Matriz SPSD 4×4)

Consideremos la siguiente matriz A_0 simétrica y semidefinida positiva, y la matriz de vectores propios inicializada como la identidad $V_0 = I_4$:

$$A_0 = \begin{pmatrix} 4 & 2 & 0 & 0 \\ 2 & 4 & 0 & 0 \\ 0 & 0 & 5 & 1 \\ 0 & 0 & 1 & 5 \end{pmatrix}, \quad V_0 = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Iteración 1 ($k = 1$)

El elemento fuera de la diagonal con mayor valor absoluto es $a_{12} = 2$. Por lo tanto, pivoteamos en $p = 1, q = 2$. Dado que $a_{11} = a_{22} = 4$, el ángulo de rotación se simplifica a $\theta = \frac{\pi}{4}$. Calculamos $c = \cos(\pi/4) = \frac{\sqrt{2}}{2}$ y $s = \sin(\pi/4) = \frac{\sqrt{2}}{2}$. La matriz de rotación ortogonal P_1 se define como:

$$P_1 = \begin{pmatrix} c & -s & 0 & 0 \\ s & c & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Actualizamos $A_1 = P_1^T A_0 P_1$ y $V_1 = V_0 P_1$:

$$A_1 = \begin{pmatrix} 6 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 5 & 1 \\ 0 & 0 & 1 & 5 \end{pmatrix}, \quad V_1 = \begin{pmatrix} c & -s & 0 & 0 \\ s & c & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Iteración 2 ($k = 2$)

Ahora el elemento máximo fuera de la diagonal en A_1 es $a_{34} = 1$. Pivoteamos en $p = 3, q = 4$. Como $a_{33} = a_{44} = 5$, el ángulo es nuevamente $\theta = \frac{\pi}{4}$. Utilizando los mismos $c = \frac{\sqrt{2}}{2}, s = \frac{\sqrt{2}}{2}$, la nueva matriz P_2 es:

$$P_2 = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & c & -s \\ 0 & 0 & s & c \end{pmatrix}$$

Actualizamos $A_2 = P_2^T A_1 P_2$ y $V_2 = V_1 P_2$:

$$A_2 = \begin{pmatrix} 6 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 6 & 0 \\ 0 & 0 & 0 & 4 \end{pmatrix}, \quad V_2 = \begin{pmatrix} c & -s & 0 & 0 \\ s & c & 0 & 0 \\ 0 & 0 & c & -s \\ 0 & 0 & s & c \end{pmatrix}$$

Conclusión de la ejecución

En A_2, todos los elementos fuera de la diagonal principal son cero. El algoritmo converge.

- **Valores propios:** Se extraen de la diagonal de A_2: $\lambda = \{6, 2, 6, 4\}$.
- **Vectores propios:** Están contenidos en las columnas de V_2.

Finalmente, aplicando la regla de ordenamiento de mayor a menor, los valores propios se ordenan como $\lambda = (6, 6, 4, 2)^T$ y las columnas de V_2 se reorganizan de manera congruente para coincidir con el orden final.

Referencias

[1] G. H. Golub y C. F. Van Loan, *Matrix Computations*, 4ta ed. Baltimore, MD, EE. UU.: Johns Hopkins University Press, 2013.

[2] R. L. Burden y J. D. Faires, *Numerical Analysis*, 9na ed. Boston, MA, EE. UU.: Brooks/Cole, 2011.

[3] S. C. Chapra y R. P. Canale, *Numerical Methods for Engineers*, 7ma ed. Nueva York, NY, EE. UU.: McGraw-Hill Education, 2015.